

LECTURE SERIES · ISEA*Intelligent Digital Forensics: LLMs, NLP, and Explainable AI for Cyber Investigations***Session 5 Lab Manual****Digital Forensics in Google Colab: from your first cell to an explainable verdict**

Seven hands-on activities. No coding background assumed. Every step is written out, every command is explained, and every expected output shown here was produced by actually running the code (nothing is invented)

Companion files: ISEA_S5_Colab_Forensics_Lab.ipynb (the notebook this manual walks through) and the Session 5 slide deck.

Dataset: SMS Spam Collection (Almeida & Gómez Hidalgo), UCI Machine Learning Repository with 5,574 real, labelled SMS messages.

<https://archive.ics.uci.edu/dataset/228/sms+spam+collection>

How to use this manual

Each activity is a self-contained lab exercise. In the live session, we run a subset together (the deck tells you which); the rest are for self-paced work. The manual assumes nothing beyond a Google account and a browser.

Three conventions:

- **Dark blocks** are code you type (or paste) into a Colab code cell and run.
- **Light blocks** are the output you should see. Your numbers should match; tiny differences in the last decimal can occur if library versions differ.
- **Boxes:** navy = why the step matters · green = checkpoint before continuing · amber = optional challenge.

The one rule of the bench: run cells top to bottom, in order. The bracketed number left of a cell [4], records the order it actually ran. In forensic work, an analysis you cannot re-run in order is an analysis you cannot defend.

Part I · The Bench

Google Colab from zero: what it is, how to drive it, and how to install the forensic toolkit.

Activity 1 · First contact with Google Colab

Time: ≈ 45 min **Needs:** a Google account, any browser **You produce:** your first saved notebook

What Colab is, in one paragraph: Google Colaboratory gives you a temporary Linux computer in a Google data centre, driven from your browser through a **notebook**, a document that mixes runnable code, its output, and ordinary text. Nothing installs on your machine, the free tier is enough for everything in this lab, and the notebook file itself lives in your Google Drive. For an examiner, one property matters above all: a notebook is a *contemporaneous record* that includes the code, the order it ran, and what it produced, in one reviewable artefact.

Step 1. Open colab.research.google.com and sign in with your Google account. You land on a welcome screen with an “Open notebook” dialog.

Step 2. Choose **New notebook** (bottom-left of the dialog, or File ▶ New notebook in Drive). A blank notebook opens named Untitled0.ipynb.

Step 3. Rename it: click the title at the top-left and type ISEA_S5_practice.ipynb. Colab autosaves to a Drive folder called Colab Notebooks (you never press Save).

Step 4. Tour the room before touching anything. Find these five things: **(a)** the **+ Code** and **+ Text** buttons under the menu bar; **(b)** the round ▶ **run button** that appears at the left of a cell when you hover; **(c)** the **Connect** button at the top-right (it becomes a RAM/Disk meter once a machine is attached); **(d)** the **folder icon** in the left sidebar, the temporary machine's file browser; **(e)** the **Runtime** menu (you will need it when things go wrong).

Step 5. Click into the empty cell, type the line below, and press **Shift+Enter**.

```
print("Hello. This machine is not my laptop.")
```

Expected output

```
Hello. This machine is not my laptop.
```

What just happened, and why it matters. Two things occurred. First, Colab attached a virtual machine to your notebook; watch the top-right corner change from “Connect” to a green tick with RAM and Disk gauges. Second, the bracket left of the cell now reads [1]: this cell was the first to run. That counter is your audit trail.

Step 6. Learn the three ways to run a cell, try each: **Shift+Enter** runs and moves to the next cell; **Ctrl+Enter** runs and stays; **Alt+Enter** runs and inserts a fresh cell below.

Step 7. Add a text cell: click **+ Text**, and type ## Case notes then, on the next line, Examiner: <your name>. Click away the ## becomes a heading. Text cells use Markdown; # marks headings, ****bold**** marks bold. This is where a real notebook carries its narrative: what you did and why.

Step 8. Break the machine on purpose. Open the **Runtime** menu and choose **Restart session**. Confirm. Now click your first cell and run it again, it works. Then note what changed: the

counter is back to [1]. A restart wipes the machine's memory, every variable, every loaded file but your **text and code survive**, because they live in the notebook, not the machine.

Step 9.

✓ **CHECKPOINT** — You can create, rename, and run a notebook; you know Shift+Enter; you have restarted a runtime and understood what it forgets (memory) and what it keeps (your notebook).

The three facts about the temporary machine

- **It is temporary.** Idle too long (typically on the order of an hour and a half on the free tier) and it is reclaimed; there is also a daily ceiling per session. Files you downloaded onto it vanish with it; the notebook itself never does.
- **It is a full Linux computer.** Lines starting with ! are commands to the machine, not Python: !ls lists its files, !pip install adds software to it.
- **CPU is enough for this lab.** Runtime ► Change runtime type shows GPU options — leave them alone today; scarce accelerators are for when models get heavy.

Try it yourself. In your practice notebook, run !ls / and !cat /etc/os-release — meet the Linux system you have borrowed. Then upload any small text file using the folder sidebar's upload button, run !ls to see it, restart the runtime, and run !ls again. It is gone. That lesson costs nothing today; unlearned, it costs evidence later.

Activity 2 · Just enough Python

Time: ≈ 30 min **Needs:** Activity 1 **You produce:** the ability to read the code in this lab

This lab needs you to read Python, not to be fluent in it. Five ideas cover everything you will see. Type each block into a fresh cell in your practice notebook and run it (preferably typing, not pasting, is where the learning happens at this stage).

Step 1. Variables — labelled boxes for values.

```
investigator = "A. Sharma"
case_id = "ISEA-S5-001"
messages_seized = 5574
print(investigator, "| case:", case_id, "| items:", messages_seized)
```

Expected output

```
A. Sharma | case: ISEA-S5-001 | items: 5574
```

Step 2. Strings — text in quotes, with tools attached. A dot reaches those tools.

```
msg = "WIN a FREE prize NOW"
print(msg.lower())
print("free" in msg.lower()) # 'in' asks: does it contain this?
```

Expected output

```
win a free prize now
True
```

Step 3. Lists and loops — several values, visited one at a time.

```
evidence = ["msg_001.txt", "msg_002.txt", "msg_003.txt"]
for item in evidence:
    print("examining", item)
```

Expected output

```
examining msg_001.txt
examining msg_002.txt
```

```
examining msg_003.txt
```

What just happened, and why it matters. The four leading spaces on the print line are not decoration — indentation is how Python knows which lines belong inside the loop. Delete them and the code changes meaning or breaks.

Step 4. If — decisions.

```
score = 0.97
if score > 0.9:
    print("flag for review")
else:
    print("release")
```

Expected output

```
flag for review
```

Step 5. Imports — bringing in toolkits others built. Everything serious in this lab arrives this way.

```
import pandas as pd    # data tables; 'pd' is its customary short name
print(pd.__version__)
```

Expected output

```
2.2.2 (any version number is fine)
```

Step 6. Now read an error properly — run this, and let it fail:

```
print(evidence_count)
```

Expected error

```
NameError: name 'evidence_count' is not defined
(last line of a longer red message)
```

What just happened, and why it matters. Python errors read bottom-up: the **last line is the diagnosis** (here: you used a name that does not exist — usually a typo, or a cell you forgot to run), and the lines above it are the trail. Every red wall of text you will ever see ends in one plain sentence. Read that sentence first.

✓ **CHECKPOINT** — You can predict what a variable, a loop, an if, and an import do — and when a cell fails, you read the last line before anything else.

Try it yourself. Write a loop over ["FREE offer!!", "see you at 5", "URGENT verify now"] that prints only the messages containing “urgent” or “free”, in any letter-case. (Hint: .lower() and in.)

Activity 3 · Set up the forensic toolkit

Time: ≈ 15 min **Needs:** Activity 1 **You produce:** a ready bench — Section 0 of the lab notebook, green From here on, work in the real lab notebook. Open the distributed ISEA_S5_Colab_Forensics_Lab.ipynb link and immediately do **File ▶ Save a copy in Drive**, otherwise you are editing a read-only original and your work will not persist. Work in your copy.

Step 1. Run the first Section 0 cell:

```
!pip install -q lime shap
print("Libraries installed.")
```

Expected output

```
Libraries installed.
(pip may print version-resolution lines above this — normal)
```

What just happened, and why it matters. Colab machines arrive with the heavy tools such as pandas, scikit-learn, NLTK, spaCy, matplotlib already installed. We add only the two explanation libraries.

What is pip?

pip is Python's **package manager**.

Think of it like an app store for Python.

- Your phone uses **Play Store** or **App Store** to install apps.
- Python uses **pip** to install libraries (packages).
- Google Colab already comes with many libraries installed, but not all of them.
- If your notebook needs a library that isn't installed (or needs a newer version), you use pip.

The `-q` keeps pip quiet. If pip ever warns that you “may need to restart the runtime”, do it (Runtime ► Restart session), then re-run Section 0 — installed packages survive a restart; loaded ones do not.

Why is there an exclamation mark (!)?

This is the part that often confuses people.

The `!` tells Colab:

"This is not Python code. Send this command to the Linux operating system."

Without `!`, Python thinks pip is a Python variable or function.

Step 2. Run the second Section 0 cell — language resources:

```
import nltk
for pkg in ["punkt", "punkt_tab", "stopwords"]:
    nltk.download(pkg, quiet=True)

import spacy
try:
    nlp = spacy.load("en_core_web_sm")
except OSError:
    import subprocess, sys
    subprocess.run([sys.executable, "-m", "spacy", "download", "en_core_web_sm", "-q"])
    nlp = spacy.load("en_core_web_sm")

print("Bench ready.")
```

Expected output

Bench ready.

What just happened, and why it matters. NLTK ships empty and fetches what it needs by name: punkt/punkt_tab split text into words (newer NLTK versions look for the second name and downloading both makes the notebook version-proof), and stopwords is a list of filler words (the, is, at...). spaCy's en_core_web_sm is a small pretrained English model; the try/except loads it if present and downloads it only if not.

Step 3. Meet your toolkit — what each library is, in evidence terms: **pandas** organises evidence into tables · **NLTK** / **spaCy** examine language (split, filter, extract names and entities) · **scikit-learn** builds and tests the triage model · **LIME** / **SHAP** interrogate the model's decisions. Nothing else is needed today.

✓ **CHECKPOINT** — Both Section 0 cells show green ticks and “Bench ready.” If anything failed, the error table in Appendix A almost certainly has it.

Part II · The Case

A fraud team hands you an export of 5,574 SMS messages, each labelled ham (normal traffic) or spam (unsolicited/scam bait). The brief: build a first-pass triage tool that separates scam bait from normal traffic at scale and then decide whether you can trust it. This mirrors real smishing triage: the volume is too high to read everything, and the cost of a wrong call runs in both directions.

Activity 4 · Load the evidence

Time: ≈ 20 min **Needs:** Activity 3 **Notebook:** Section 2 **You produce:** the corpus, loaded and profiled **Provenance first, always.** This is the SMS Spam Collection assembled by Almeida and Gómez Hidalgo: real messages, gathered and labelled by researchers, published through the UCI Machine Learning Repository. In casework you would record who collected the data, how, and under what authority, before touching a single row. We practise the reflex even in a lab.

Step 1. Download the corpus onto the runtime machine (Section 2, first cell). The code tries UCI, and falls back to a public mirror of the same corpus if UCI is busy:

```
import requests, zipfile, io

url = "https://archive.ics.uci.edu/static/public/228/sms+spam+collection.zip"
try:
    r = requests.get(url, timeout=60)
    zipfile.ZipFile(io.BytesIO(r.content)).extractall(".")
    print("Downloaded from UCI.")
except Exception as e:
    print("UCI unreachable, using mirror:", e)
    r = requests.get("https://raw.githubusercontent.com/justmarkham/DAT8/"
                    "master/data/sms.tsv", timeout=60)
    with open("SMSSpamCollection", "w") as f:
        f.write(r.text)
    print("Downloaded from mirror.")
```

Expected output

Downloaded from UCI.

What just happened, and why it matters. The file lands on the temporary machine (check the folder sidebar, SMSSpamCollection is now there). If the runtime dies, the file dies with it; the Rescue cell in the notebook's appendix re-fetches everything in one go. Note the try/except pattern, a downloader that cannot fail politely is a downloader that strands a classroom.

Step 2. Load it into a table:

```
import pandas as pd

df = pd.read_csv("SMSSpamCollection", sep="\t", header=None,
                 names=["label", "message"], quoting=3)
print("Rows and columns:", df.shape)
df.head(8)
```

Expected output

Rows and columns: (5574, 2)

(followed by a table of the first 8 rows: a 'label' column and a 'message' column)

What just happened, and why it matters. Each argument earns its place: `sep="\t"` — the file separates label from message with a tab; `header=None` — the file has no title row, so we name the columns ourselves; `quoting=3` — treat quote marks as ordinary characters (messages

contain them; without this, rows silently merge and you lose two messages without any error). Silent data loss is the worst kind, the load step is where you stop it.

Step 3. Profile the evidence:

```
print(df["label"].value_counts())
print()
print("Spam share: %.1f%%" % (100 * (df["label"] == "spam").mean()))
```

Expected output

```
label
ham    4827
spam    747
Name: count, dtype: int64
```

```
Spam share: 13.4%
```

What just happened, and why it matters. Class imbalance of 13.4% spam is the normal condition of forensic triage: the thing you hunt is always the minority. Hold that number; it returns with force in Activity 6, when a high accuracy score turns out to be hiding something.

Step 4. Read actual messages — never model what you have not read:

```
print("--- three spam messages ---")
for m in df[df.label == "spam"]["message"].head(3):
    print("•", m, "\n")
print("--- three ham messages ---")
for m in df[df.label == "ham"]["message"].head(3):
    print("•", m, "\n")
```

Expected output

```
--- three spam messages ---
• Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. ...
(two more)
--- three ham messages ---
• Go until jurong point, crazy.. Available only in bugis n great world la e ...
(two more)
```

✓ **CHECKPOINT** — 5,574 rows; 4,827 ham / 747 spam; you have read at least three of each with your own eyes and could describe, in words, how they differ.

Try it yourself. Show the ten longest messages: `df.assign(n=df.message.str.len()).sort_values("n", ascending=False).head(10)`. Are the longest messages spam or ham? Form a hypothesis before you look.

Activity 5 · Indicators before models

Time: ≈ 30 min **Needs:** Activity 4 **Notebook:** Section 3 **You produce:** an indicator profile of the corpus — and Session 2's debt, paid

Before any machine learning: what do scam messages contain that normal ones do not? We count four indicator types with pattern matching, the way an examiner greps a log and then bring in NLTK and spaCy for the language-level view.

Step 1. Run the indicator scan:

```
indicators = {
    "contains URL":      r"(?:https?://|www\.)\S+",
    "long number (5-11 digits)": r"\b\d{5,11}\b",
    "money symbol/amount":  r"[\£$€]\s?\d[\d\.\,]*|\b\d+\s?p\b",
    "bait word":          r"\b(?:free|win|winner|urgent|claim|prize)\b",
}
```

```
print(f'{"indicator":28s}{in spam':>10s}{in ham':>10s}")
for name, pattern in indicators.items():
    hit = df["message"].str.contains(pattern, regex=True, case=False)
    print(f'{"name":28s}{100*hit[df.label=="spam"].mean():9.1f}%
          f'{"100*hit[df.label=="ham"].mean():9.1f}%")
```

Expected output

indicator	in spam	in ham
contains URL	14.2%	0.0%
long number (5-11 digits)	74.6%	0.1%
money symbol/amount	50.6%	0.2%
bait word	49.7%	1.5%

What just happened, and why it matters. Those patterns are regular expressions, a compact language for “text shaped like this”: `\d{5,11}` means 5-to-11 digits in a row (callback numbers, premium shortcodes, claim codes); `[£$€]` means any one of those currency symbols. And look at the numbers: three-quarters of scams carry a long number; almost no normal message does. You have real, articulable signal **before any model exists**, remember this when the model later takes the credit.

Step 2. Tokenise one real scam with NLTK, and strip the filler:

```
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

evidence_msg = ("You have WON a guaranteed £1000 cash or a £2000 prize. "
               "To claim yr prize call our customer service representative "
               "on 09061701461.")

tokens = word_tokenize(evidence_msg)
print("Tokens:", tokens[:14], "...")

sw = set(stopwords.words("english"))
content = [t for t in tokens if t.lower() not in sw and t.isalpha()]
print("Content words only:", content)
```

Expected output

Tokens: ['You', 'have', 'WON', 'a', 'guaranteed', '£1000', 'cash', 'or', 'a', '£2000', 'prize', '.', 'To', 'claim'] ...
 Content words only: ['guaranteed', 'cash', 'prize', 'claim', 'yr', 'prize', 'call', 'customer', 'service', 'representative']

What just happened, and why it matters. Tokenisation is splitting text into units. It is the first step of every NLP pipeline you will ever run. Stopword removal keeps the load-bearing words. Ten content words, and the scam’s whole grammar is visible: a promise (guaranteed, cash, prize), an action (claim, call), and a costume (customer service representative).

Step 3. Now spaCy’s named-entity recognition on the same message and read the output critically:

```
doc = nlp(evidence_msg)
for ent in doc.ents:
    print(f'{"ent.text:22s"} -> {"ent.label_}")
```

Expected output

1000	-> MONEY
2000	-> MONEY
09061701461	-> DATE

What just happened, and why it matters. Read the last line again. spaCy tagged 09061701461 — the premium-rate callback number, the operational heart of this scam — as a **DATE**. The model was trained on news text; SMS-speak confuses it. Nothing is broken: this is what a

pretrained model does outside its home domain. The sentence to keep, verbatim, from Session 2's objectives: an NLP tool's output is a lead to verify, never a fact to file.

✓ **CHECKPOINT** — You have the four indicator rates, you can explain what `\d{5,11}` matches, and you can say — with the example on the screen — why NER output goes into a report only after a human checked it.

Try it yourself. Add a fifth indicator for messages written mostly in capitals, e.g. `r"\b[A-Z]{2,}\b"` with `.str.count()` instead of `contains` — spam averages 2.04 all-caps words per message against 0.42 for ham. Then try `spaCy` on two more spam messages of your choosing: how often are entities right?

Activity 6 · Train and judge the triage model

Time: ≈ 30 min **Needs:** Activity 5 **Notebook:** Sections 4-5 **You produce:** a working classifier — and the vote

Two moves. First, turn each message into numbers with **TF-IDF**: each message becomes a long scorecard of which words it uses, weighted so that distinctive words (prize, claim) count more than ubiquitous ones. Second, fit **logistic regression**, a simple classifier that learns one weight per word and adds evidence up. We choose it deliberately: it is fast, strong on text, and unlike deeper models you can open it and look inside. That property becomes the whole story in Part III.

Step 1. Split, then train (Section 4). The 25% we hold out is never shown to the model, it is the only honest test:

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

X_train, X_test, y_train, y_test = train_test_split(
    df["message"], df["label"],
    test_size=0.25, random_state=42, stratify=df["label"])
print("Training messages:", len(X_train), "| Held-out test messages:", len(X_test))

model = make_pipeline(
    TfidfVectorizer(lowercase=True, stop_words="english"),
    LogisticRegression(max_iter=1000, random_state=42))
model.fit(X_train, y_train)
print("Model trained.")
```

Expected output

```
Training messages: 4180 | Held-out test messages: 1394
Model trained.
```

What just happened, and why it matters. Three arguments matter: `random_state=42` fixes the shuffle so anyone can reproduce your exact split — reproducibility is a forensic requirement, not a style choice; `stratify` keeps the 13.4% spam share identical in both halves; and the pipeline welds vectoriser and classifier together so raw text goes in and a verdict comes out.

Step 2. Evaluate and read it like an examiner, not a leaderboard:

```
from sklearn.metrics import (accuracy_score, classification_report,
                             ConfusionMatrixDisplay)
import matplotlib.pyplot as plt

pred = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, pred)*100:.2f}%\n")
```

```
print(classification_report(y_test, pred, digits=3))

ConfusionMatrixDisplay.from_predictions(
    y_test, pred, labels=["ham", "spam"],
    display_labels=["ham (normal)", "spam (scam)"], cmap="Blues", colorbar=False)
plt.title("Held-out test set")
plt.show()
```

Expected output

Accuracy: 97.27%

	precision	recall	f1-score	support
ham	0.969	1.000	0.985	1207
spam	1.000	0.797	0.887	187

(confusion-matrix figure below)

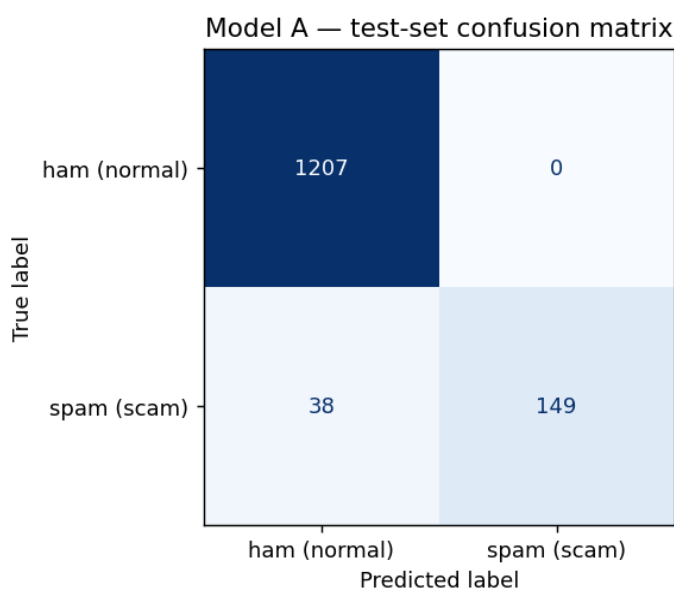


Figure 1 — the confusion matrix your cell produces. Top-left and bottom-right are correct calls; the off-diagonal cells are the two kinds of mistake.

What just happened, and why it matters. The headline says 97.27%. The matrix says more: **false alarms — 0** (no innocent message was accused; spam precision 1.000) but **misses 38 of 187** (spam recall 0.797; that is one scam in five walks straight through). Same model, same numbers, two different stories. In triage, the two errors have different victims: a false alarm wastes an analyst's minute; a miss reaches someone's parents. Accuracy is a headline; the confusion matrix is the finding. And notice the mechanism of the flattery: with 86.6% of traffic being ham, a model can score in the nineties while fumbling the minority class — the imbalance from Activity 4, cashing out.

Step 3. The verdict machine (Section 5) — score messages it has never seen:

```
new_messages = [
    "URGENT! You have won a free prize of £2000. Call 09061701461 now to claim.",
    "ok bro, reaching campus by 5, wait near the gate",
]
proba = model.predict_proba(new_messages)
spam_col = list(model.classes_).index("spam")
for msg, p in zip(new_messages, proba):
    print(f"p(spam) = {p[spam_col]*100:5.1f}% {msg}")
```

Expected output

```
p(spam) = 91.9% URGENT! You have won a free prize of £2000. Call 09061701461 now to claim.
p(spam) = 3.8% ok bro, reaching campus by 5, wait near the gate
```

Step 4. Find a real held-out message the model is ~97% sure about — this becomes “the alert” for Part III:

```
import numpy as np
p_test = model.predict_proba(X_test)[:, spam_col]
i = int(np.argsort(np.abs(p_test - 0.97))[0])
alert_msg = X_test.iloc[i]
print(f"p(spam) = {p_test[i]*100:.1f}%")
print("MESSAGE:", alert_msg)
```

Expected output

```
p(spam) = 96.7%
MESSAGE: Get ur 1st RINGTONE FREE NOW! Reply to this msg with TONE. Gr8 TOP 20 tones to your phone ...
```

STOP HERE — the vote. Your model — the one you trained five minutes ago — says ~97% spam on that message, with a queue behind it and no explanation attached. Write in your worksheet, before scrolling further: **ACT / DON'T / NEED MORE** — and if NEED MORE, exactly what you would need to see. Everything in Part III is the answer to whatever you just wrote.

✓ **CHECKPOINT** — 97.27% accuracy, 0 false alarms, 38 misses; two fresh messages scored; the alert message found at 96.7%; your vote is written down.

Try it yourself. Write three messages of your own — one obvious scam, one obviously normal, one deliberately ambiguous (e.g. a genuine bank alert) — and score them. Where does the model put your ambiguous one, and do you agree with it?

Part III · The Interrogation

The model gives verdicts; it does not give reasons. This part is the interrogation: LIME asks the model questions from outside, SHAP splits the credit for a decision by a fairness rule, and the audit in Activity 8 shows why none of this is decoration — it is the only instrument that catches a model that is right for the wrong reason.

Activity 7 · Explain the verdict — LIME and SHAP

Time: ≈ 40 min **Needs:** Activity 6 **Notebook:** Sections 6-7 **You produce:** Exercises 1 and 2 in the worksheet

Part A — LIME: ask the black box what it would have said

LIME never looks inside the model. It perturbs this one message hundreds of times — dropping words — watches the score move, and fits a small, readable model to that neighbourhood. What you get: the words that pushed this prediction, with direction and weight.

Step 1. Explain the alert message (Section 6):

```
from lime.lime_text import LimeTextExplainer

explainer = LimeTextExplainer(class_names=list(model.classes_), random_state=42)
exp = explainer.explain_instance(alert_msg, model.predict_proba,
                                num_features=8, num_samples=1000)

from IPython.display import HTML
HTML(exp.as_html())
```

Expected output

(an interactive panel: class probabilities, a bar chart of word weights, and the message with the influential words highlighted)

Step 2. The same result as plain numbers — positive pushed towards spam:

```
for word, weight in exp.as_list():
    print(f'{word:14s} {weight:+.3f}')
```

Expected output

```
STOP      +0.083
Reply     +0.068
FREE      +0.066
50        +0.051
RINGTONE  +0.047
16        +0.042
TONE      +0.039
msg       +0.034
```

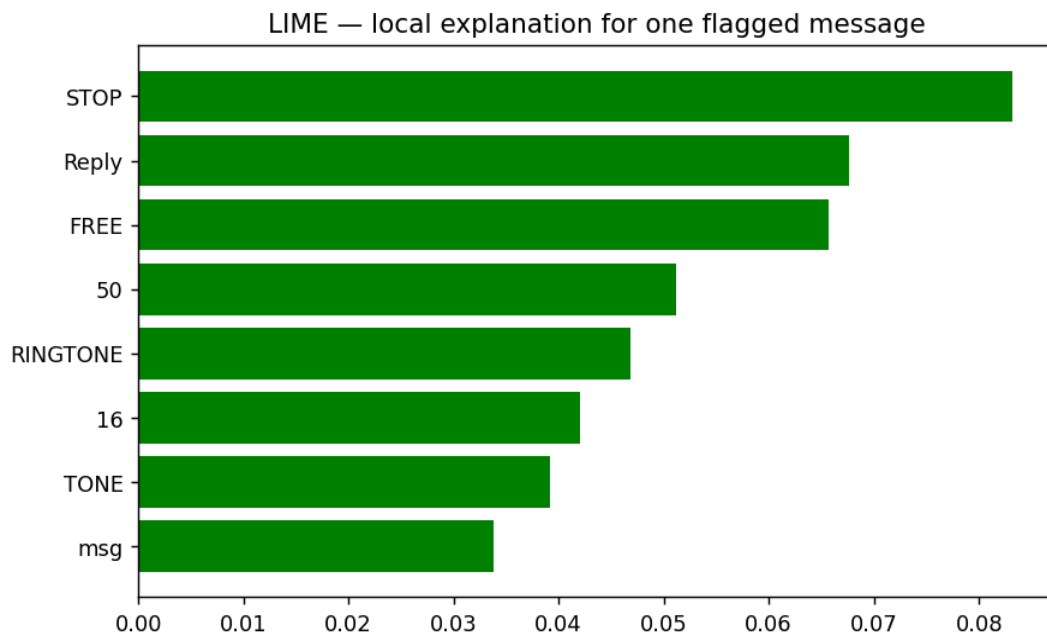


Figure 2 — LIME's local explanation for the alert message, as produced by `exp.as_pyplot_figure()`. Every bar is a word; length is influence on this one verdict.

What just happened, and why it matters. Read the top row twice. ‘STOP’ opt-out wording that bulk senders are required to include is the strongest single push towards “scam”. The model has learned **compliance boilerplate** as its best scam marker. Is that evidence about the message, or about the sender's mailing software? Neither answer is comfortable, and that discomfort is exactly what an explanation is for: it turns “the system said so” into a claim you can argue with.

Step 3. Stability test — different seed, a fifth of the samples:

```
exp2 = LimeTextExplainer(class_names=list(model.classes_), random_state=7) \
    .explain_instance(alert_msg, model.predict_proba,
                     num_features=8, num_samples=200)
for word, weight in exp2.as_list():
    print(f"{word:14s} {weight:+.3f}")

top1 = {w for w, _ in exp.as_list()[:5]}
top2 = {w for w, _ in exp2.as_list()[:5]}
print("\nTop-5 overlap:", len(top1 & top2), "of 5. Changed:", top1 ^ top2)
```

Expected output

```
STOP      +0.077
Reply     +0.072
FREE      +0.070
50        +0.053
16        +0.043
RINGTONE  +0.041
TONE      +0.040
Gr8       -0.021
```

```
Top-5 overlap: 4 of 5. Changed: {'RINGTONE', '16'}
```

What just happened, and why it matters. One of five top features swapped when the seed and sample size changed. Not a bug but the approximation cost of a sampled method. The rule to carry: **if a ranking is not stable, you have a measurement, not a finding.** Report the seed and sample size, or do not report the ranking.

Part B — SHAP: credit split by a fairness rule

SHAP borrows from game theory: features are players, the prediction is the payout, and each feature's value is its fair share of the credit. The values sum exactly to the gap between this prediction and the model's baseline which is why SHAP plots reconcile and LIME's do not.

Step 4. Compute SHAP values for every held-out message (Section 7):

```
import shap

vec = model.named_steps["tfidfvectorizer"]
clf = model.named_steps["logisticregression"]
X_train_v = vec.transform(X_train)
X_test_v = vec.transform(X_test)
feature_names = list(vec.get_feature_names_out())

shap_explainer = shap.LinearExplainer(clf, X_train_v)
sv = shap_explainer(X_test_v)
sv.feature_names = feature_names
print("SHAP values computed for", sv.values.shape[0], "test messages ×",
      sv.values.shape[1], "features")
```

Expected output

```
SHAP values computed for 1394 test messages × 7184 features
(a note about subsampling the background to 100 rows may appear — that is the
approximation cost from the lecture, stated out loud by the library)
```

Step 5. Local — the waterfall for the alert message:

```
one = shap.Explanation(
    values=np.asarray(sv.values[i].todense()).ravel()
    if hasattr(sv.values[i], "todense") else sv.values[i],
    base_values=float(sv.base_values[i]) if np.ndim(sv.base_values)
    else float(sv.base_values),
    data=np.asarray(X_test_v[i].todense()).ravel(),
    feature_names=feature_names)
shap.plots.waterfall(one, max_display=10)
```

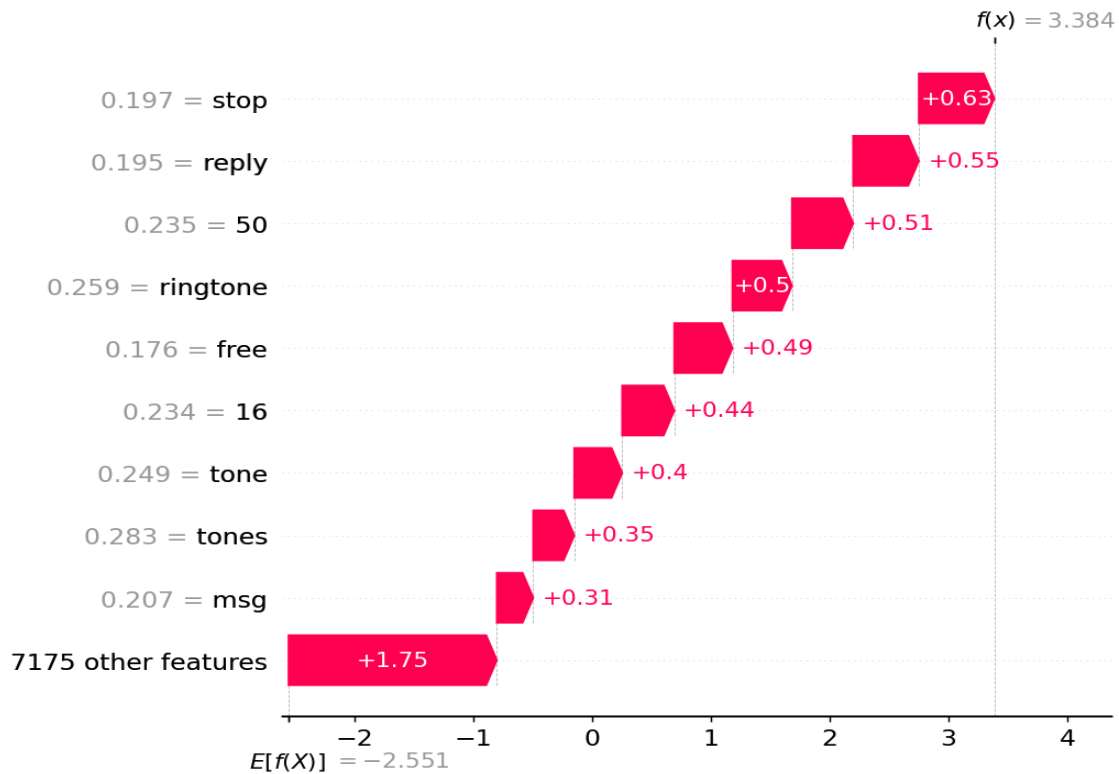


Figure 3 — SHAP waterfall for the alert message: starts at the model's baseline, adds each word's push, lands on the final score. Answers: why this one?

Step 6. Global — the beeswarm across a 300-message sample:

```
rng = np.random.RandomState(0)
samp = rng.choice(X_test_v.shape[0], 300, replace=False)
vals = sv.values[samp]
vals = np.asarray(vals.todense()) if hasattr(vals, "todense") else vals
sub = shap.Explanation(
    values=vals,
    base_values=np.full(300, float(sv.base_values
        if np.ndim(sv.base_values) == 0 else sv.base_values[0])),
    data=np.asarray(X_test_v[samp].todense()),
    feature_names=feature_names)
shap.plots.beeswarm(sub, max_display=12)
```

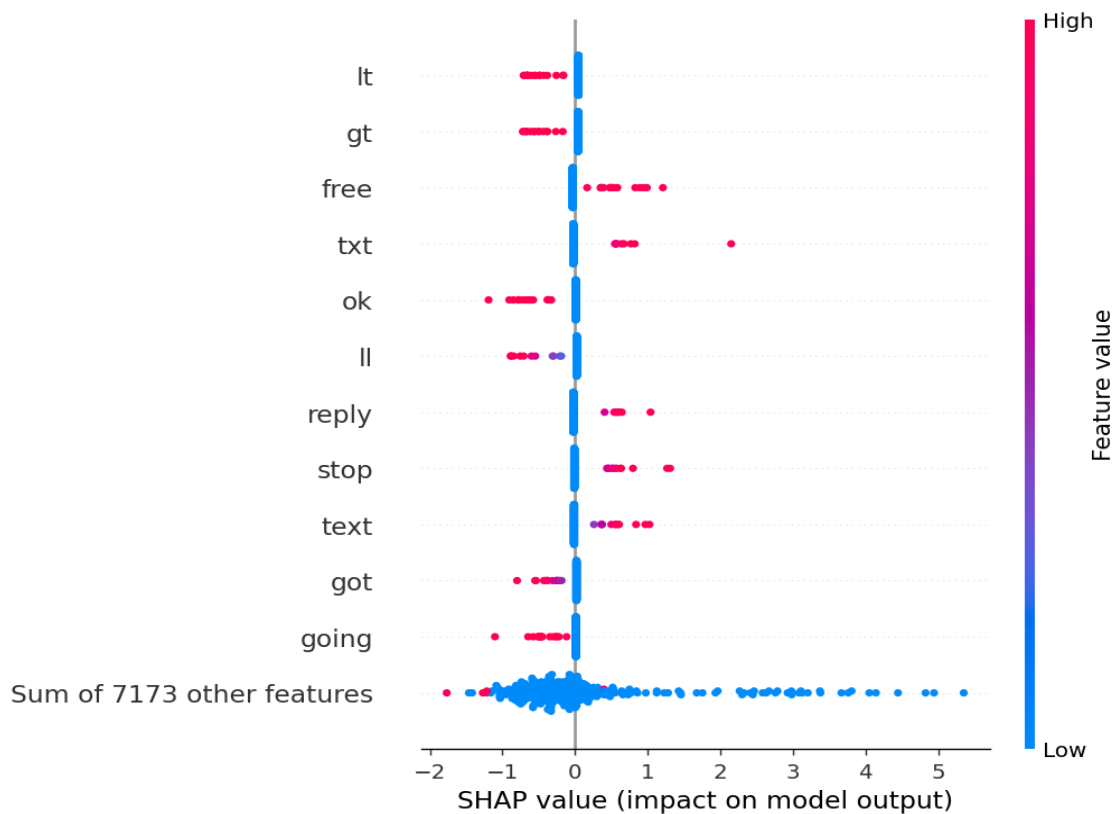


Figure 4 — the beeswarm: every sampled message, every feature. Answers: what does this model run on, overall?

What just happened, and why it matters. Look at the beeswarm's top rows. It and gt are fragments of < and > HTML escape codes left behind by how this corpus was anonymised and prepared. Two of your model's most influential features are **packaging artefacts, not language**. Nobody planted this: it was sitting in a standard research corpus, and no accuracy number would ever have told you. The global explanation surfaced it in one glance.

✓ **CHECKPOINT** — Exercise 1: LIME words you would and would not defend, plus your stability result (4 of 5, and what moved). Exercise 2: the features LIME and SHAP agree on, and the honest sentence — “the model's decision on this message was driven principally by ___ and ___”. Model behaviour, not world causation.

Try it yourself. Explain a message the model got wrong, one of the 38 misses: `miss = X_test[(y_test == "spam") & (pred == "ham")].iloc[0]` and then run LIME on it. What was missing from the message that the model apparently needed?

Appendix A · When something breaks

Every error below was either produced on purpose while writing this lab or is a standard Colab failure. Read the last line of the red output, then find it here.

If you see...	It means / do this
NameError: name '...' is not defined	You used a name before creating it — usually a cell above was never run, or was run after a restart wiped memory. Run the cells above in order (or Runtime ► Run all), or use the notebook's Rescue cell.
ModuleNotFoundError: No module named 'lime'	The install cell has not run on this machine. Re-run Section 0. If pip asked for a restart: Runtime ► Restart session, then Section 0 again.
OSError: [E050] Can't find model 'en_core_web_sm'	spaCy's model is missing — run the second Section 0 cell; its try/except downloads it.
LookupError: Resource punkt_tab not found	NLTK's tokeniser data is missing — run the second Section 0 cell (it downloads punkt, punkt_tab, and stopwords).
FileNotFoundError: 'SMSSpamCollection'	The dataset is not on this machine (fresh or restarted runtime). Re-run the download cell in Section 2, or the Rescue cell.
SyntaxError: invalid syntax	Python could not read the line — most often a missing quote, bracket, or colon, or a pasted line that lost its indentation. Compare character-by-character with the manual.
IndentationError: expected an indented block	A line after for/if/def must be indented (four spaces). Colab indents for you when you end a line with a colon — do not delete it.
Cell runs forever (spinning icon)	Interrupt it: Runtime ► Interrupt execution. If the machine is truly stuck: Runtime ► Restart session, then the Rescue cell.
"Runtime disconnected" banner	Idle timeout or a network blip. Click Reconnect; if variables are gone (NameErrors follow), run the Rescue cell — it rebuilds the whole bench in ≈2 minutes.
Your numbers differ in the 2nd decimal	Minor library-version drift — acceptable. If they differ wildly, check random_state=42 survived in your copy of the cell.

Appendix B · Keyboard survival card

- **Shift+Enter** — run cell, move to next • **Ctrl+Enter** — run cell, stay • **Alt+Enter** — run cell, insert new one below
- **Ctrl+M B** — new code cell below • **Ctrl+M D** — delete current cell • **Ctrl+M Z** — undo cell action
- **Ctrl+F9** — run all cells • **Ctrl+F8** — run all cells above the current one (the honest way to rebuild state)
- **Ctrl+/** — comment or uncomment the selected lines

Appendix C · Glossary in plain words

- **Notebook (.ipynb)** — A document of runnable code cells, their outputs, and text, your contemporaneous lab record.
- **Runtime** — The temporary Linux machine attached to your notebook. It forgets everything when restarted or reclaimed; the notebook does not.
- **Cell / execution counter** — One runnable block; the bracketed number records the order it actually ran.
- **Corpus** — A body of text collected for analysis — here, 5,574 labelled SMS messages.
- **Regular expression (regex)** — A pattern language for text: `\d{5,11}` means “5 to 11 digits in a row”.
- **Token / tokenisation** — A unit of text (roughly a word); splitting text into tokens is NLP's first step.
- **Stopwords** — Filler words (the, is, at) removed to expose content words.
- **NER** — Named-entity recognition — a model that labels spans as PERSON, MONEY, DATE... a lead to verify, never a fact to file.
- **TF-IDF** — Turns a message into numbers: which words it uses, weighted by how distinctive each word is across the corpus.
- **Logistic regression** — A simple classifier: one learned weight per word, evidence added up — inspectable by design.
- **Train/test split** — Learn on one part, judge only on the untouched part — the held-out set is the only honest test.
- **Precision / recall** — Of what I flagged, how much was right? / Of what was truly there, how much did I catch? — the two errors have different victims.
- **Confusion matrix** — The 2×2 table of correct calls and both mistake types — the finding behind the accuracy headline.
- **LIME** — Explains one prediction by perturbing the input and fitting a small readable stand-in model locally.
- **SHAP** — Splits the credit for a prediction across features by a fairness rule; contributions sum exactly to the score.
- **Local vs global explanation** — Why did it flag this one? vs What does this model rely on overall? An investigation needs both.

Appendix D · Sources and where to go next

- **Dataset:** Almeida, T. A. & Gómez Hidalgo, J. M. — SMS Spam Collection, UCI Machine Learning Repository. Real messages; see the repository page for collection details and the original paper.
- **LIME:** Ribeiro, Singh & Guestrin, “Why Should I Trust You?": Explaining the Predictions of Any Classifier, KDD 2016.
- **SHAP:** Lundberg & Lee, A Unified Approach to Interpreting Model Predictions, NeurIPS 2017 (building on Shapley, 1953).
- **Clever Hans:** Lapuschkin et al., Unmasking Clever Hans predictors and assessing what machines really learn, Nature Communications 2019.
- **Fooling explainers:** Slack et al., Fooling LIME and SHAP, AIES 2020 — in security, assume the adversary reads your explainer too.
- **Practice next:** swap the corpus. The same notebook, pointed at a phishing-email corpus or your own exported chat data (with consent), is a genuine mini-project — and everything from Section 4 onward runs unchanged.